4. Factory Method Design Pattern in Java

4.1 Introduction

The **Factory Method Design Pattern** is a **creational pattern** from the Gang of Four (GoF) that defines an interface for creating objects, but lets subclasses alter the type of objects that will be created.

It promotes loose coupling and adheres to the Open/Closed Principle.

4.2 Problem with Simple Factory

Simple factories are easy to use but violate the **Open/Closed Principle**. Every time a new type is added, the factory needs to be modified.

Factory Method solves this by:

- Delegating the instantiation to subclasses
- Allowing easy extension without modification

4.3 What is Factory Method Pattern?

The Builder pattern solves this by:

- Creating a static inner class Builder
- Providing setter methods that return the builder itself (for chaining)
- A final build() method that creates the actual object

4.4 Basic Implementation Structure

Example1:

```
package com.mannemlokesh;

//Product Interface
public interface Product {
	void use();
}
```

```
package com.mannemlokesh;

//Concrete Products
public class ConcreteProductA implements Product {
	public void use() {
		System.out.println("Using Product A");
	}
}
```

```
package com.mannemlokesh;

public class ConcreteProductB implements Product {
	public void use() {
		System.out.println("Using Product B");
	}
```

```
}
```

```
package com.mannemlokesh;

//Creator (Abstract Class)
public abstract class Creator {
	public abstract Product factoryMethod();

	public void doSomething() {
		Product product = factoryMethod();
		product.use();
	}
}
```

```
package com.mannemlokesh;

//Concrete Creators
public class CreatorA extends Creator {
	public Product factoryMethod() {
		return new ConcreteProductA();
	}
}
```

```
package com.mannemlokesh;

public class CreatorB extends Creator {
	public Product factoryMethod() {
		return new ConcreteProductB();
	}
}
```

```
package com.mannemlokesh;

//Main
public class FactoryMethodDemo {
	public static void main(String[] args) {
		Creator creator = new CreatorA();
		creator.doSomething();

		creator = new CreatorB();
		creator.doSomething();
	}
}
```

Output:

```
Using Product A
Using Product B
```

4.5 Step-by-Step Examples

Example2: Beginner: Transport Factory

```
package com.mannemlokesh;

//Product
interface Transport {
```

```
	void deliver();
}

//Concrete Products
class Truck implements Transport {
	public void deliver() {
		System.out.println("Deliver by land in a box");
	}
}

class Ship implements Transport {
	public void deliver() {
		System.out.println("Deliver by sea in a container");
	}
}

//Creator
abstract class Logistics {
	public abstract Transport createTransport();

	public void planDelivery() {
		Transport t = createTransport();
		t.deliver();
	}
}

//Concrete Creators
class RoadLogistics extends Logistics {
	public Transport createTransport() {
		return new Truck();
	}
}

class SeaLogistics extends Logistics {
	public Transport createTransport() {
		return new Ship();
	}
}

//Main
public class TransportApp {
	public static void main(String[] args) {
		Logistics logistics = new RoadLogistics();
		logistics.planDelivery();

		logistics = new SeaLogistics();
		logistics.planDelivery();
	}
}
```

Output:

```
Deliver by land in a box
Deliver by sea in a container
```

Example3: Intermediate: Logger Example

```
package com.mannemlokesh;

interface Logger {
	void log(String message);
}

class FileLogger implements Logger {
	public void log(String message) {
		System.out.println("FileLogger: " + message);
	}
}

class ConsoleLogger implements Logger {
	public void log(String message) {
		System.out.println("ConsoleLogger: " + message);
	}
}

abstract class LoggerFactory {
	public abstract Logger createLogger();

	public void logMessage(String message) {
		Logger logger = createLogger();
		logger.log(message);
	}
}

class FileLoggerFactory extends LoggerFactory {
	public Logger createLogger() {
		return new FileLogger();
	}
}

class ConsoleLoggerFactory extends LoggerFactory {
	public Logger createLogger() {
		return new ConsoleLogger();
	}
}

public class LoggerApp {
	public static void main(String[] args) {
		LoggerFactory factory = new FileLoggerFactory();
		factory.logMessage("File log message");

		factory = new ConsoleLoggerFactory();
		factory.logMessage("Console log message");
	}
}
```

Output:

```
FileLogger: File log message
ConsoleLogger: Console log message
```

Example4: Advanced: GUI Framework Example

```
package com.mannemlokesh;

interface Button {
	void render();
}

class WindowsButton implements Button {
	public void render() {
		System.out.println("Rendering Windows Button");
	}
}

class MacOSButton implements Button {
	public void render() {
		System.out.println("Rendering MacOS Button");
	}
}

abstract class Dialog {
	public void renderWindow() {
		Button okButton = createButton();
		okButton.render();
	}

	public abstract Button createButton();
}

class WindowsDialog extends Dialog {
	public Button createButton() {
		return new WindowsButton();
	}
}

class MacOSDialog extends Dialog {
	public Button createButton() {
		return new MacOSButton();
	}
}

public class GuiApp {
	public static void main(String[] args) {
		Dialog dialog = new WindowsDialog();
		dialog.renderWindow();

		dialog = new MacOSDialog();
		dialog.renderWindow();
	}
}
```

Output:

```
Rendering Windows Button
Rendering MacOS Button
```

4.6 Pros and Cons

✅ **Pros:**

- Adheres to Open/Closed Principle
- Promotes decoupling and flexibility
- Easy to extend with new product types

❌ **Cons:**

- More classes and complexity
- Can lead to large hierarchies

4.7 When to Use / Not to Use

Use When:

- You need to create objects based on runtime parameters
- You want to localize object creation logic to subclasses
- You want to follow SOLID principles (OCP)

Avoid When:

- Object creation is simple and doesn't vary much
- You don’t need future extensibility

4.8 Summary

Feature	Description
Interface/Abstract	Defines factory method
Concrete Creators	Override method to return specific objects
Decoupling	Clients depend on interfaces not classes
Extendable	Add new types without changing existing code

The Factory Method Pattern is perfect for:

- UI toolkit libraries
- Loggers
- Document readers
- Enterprise frameworks

It enables flexibility, scalability, and compliance with SOLID principles.